Building Machine Learning Pipelines on the Urban Data Platform

Group 11

To support the usage of configurable machine learning pipelines in our framework, we first need to define a prediction problem in order to showcase the new ML features of the platform. For that, the problem that is being tackled in this report is the prediction of the tipping amount that will be paid by a passenger in a taxi fare. To allow the creation of new prediction problems easily, the framework only needs a JSON configuration file for each problem that defines parameters like *target* and *feature* variables, train/validation/test split percentages, and partitioning column(s) for the resulting delta table.

1 Designing and Generating the Training Dataset

To generate our training, validation, and test set, the platform uses the already-integrated enriched taxi trips (gold layer), since it contains all the information needed to make predictions (and some that is not needed). Since our prediction problem is tip estimation, the target variable is *tip_amount*, which our pipeline will predict based on the feature columns of a taxi trip.

For the features, we used all the taxi-trip columns, excluding *store_and_fwd_flag* and *congestion_surcharge* (since they do not give any information about what we are trying to predict, creating noise), and *total_amount*, since this column could "leak" tip information of the taxi ride, allowing the prediction model to take shortcuts and cheat. For the taxi zones, instead of using zone IDs, we use the corresponding borough, zone, and service names that are provided directly from the enriched table. From the weather attributes, we excluded the *wind direction*, following the decision and assumption that it is not useful in predicting the tip amount and would just create noise. Attributes *snwd* and *wpgt* were never part of the integrated gold table input, so they were skipped as well. The new columns that were introduced in the incremental update of the datasets (*humidity* and *aqi*) were also not included since they appear only in a small percentage of the rows. Finally, from the air quality columns, only the *average value of the sample measurements* was considered useful and included. Of course, in cases where there was both a pickup and a drop-off version of an attribute, both were included.

As already mentioned, all of the above columns (target and features) are defined in a JSON configuration file, tailored for this specific problem. So, the steps of the *training dataset creation job* of the framework consist of: reading the problem parameters (from the JSON), loading the wanted columns from the enriched taxi trips table, assigning a *split value* to each row (train, validation, or test - based on the given percentages), and finally storing the resulting dataset as a delta table partitioned by `pickup_borough`.

The training dataset creation job can be found in *jobs/training_set_job.py*

2 Building a Reusable Feature Engineering Pipeline

The purpose of the feature engineering pipeline is to transform and prepare a given dataset into a format suitable for machine learning.

During the design of the pipeline, we wanted to follow the same meta-driven approach as in previous weeks in order to make it as reusable and configurable as possible. At the same time, we also wanted to create a pipeline that has generic parts and functionalities as well, based on the types of the columns.

We also support transformations that are specific to each dataset. These are defined in the dataset section of the JSON configuration file. The con-

figuration can specify row-filtering operations. These dataset-specific transformations are collected and applied before the feature engineering pipeline runs.

The features are distributed into the following types and handled accordingly:

- numeric columns
- categorical columns
- datetime columns

Numeric columns are first processed by an Imputer, which fills the missing values based on the median of the other values in the same column. This is an important step to be able to handle missing data. Then the values are transformed into a vector by a VectorAssembler to make it easier to process in the last scaling step. The scaler includes the use of a StandardScaler, which standardizes the values along each column. This also makes the values easier for the model to learn.

Categorical columns are encoded according to two configurable lists in the JSON file: `onehot_columns` and `ordinal_columns`. Columns selected in either list are first converted from strings to numerical indices using a `StringIndexer`. Columns listed in `onehot_columns` are then transformed using one-hot encoding, producing a separate binary indicator for each category. Columns listed in `ordinal_columns` retain their indexed numerical representation. Finally, the resulting categorical features are included in the assembled feature vector.

Datetime columns are used to derive other date and time specific features. From each datetime column, month and day values are extracted, then in further transformations handled as categorical columns. We also apply sine and cosine transformations to the month, day of the year, and hour values. These transformations make the values circular, meaning the ends of each categorical range are brought close to each other e.g., 11 PM and 1 AM are determined to be close to each other since, in practice, there is only a 2-hour difference between the 2, but the traditional numbers do not represent it well enough. We also derive a binary `is_weekend` feature from each datetime column.

The result of the transformations is a dataset having only 3 columns

- `features`, the features put into a single vector
- `tip_amount`, the target values
- `dataset_split`, the split to which the record belongs (train/test/validation)

As described above, the pipeline supports future extensions quite well. For a new dataset, only a new configuration has to be defined for its columns, and the pipeline will be able to transform it into a feature set for training.

The detailed feature engineering analysis is presented in the evaluation report.

3 Building a Reproducible ML Pipeline

3.1 Model Training

The feature engineering pipeline built in the previous task is reused without modification. A `RandomForestRegressor` is appended as the final stage of this pipeline, so that preprocessing and model form one `Pipeline` object. The complete pipeline is fitted *only on the training split* (that is loaded from the delta table produced in 1), so when we want to predict the tip amount of a new taxi trip, the new row gets the same standardization, using the parameters of the training data. The random forest uses the same fixed seed as the data split for reproducibility. The testing and evaluation of the model is discussed in the accompanying evaluation report.

3.2 Model Persistence

Following the evaluation of the model, the fitted pipeline model (feature transformations and random forest together) is saved to `s3a://delta-data/predictions/models/tip_regressor` (path configured via the same JSON). It can later be loaded and applied to new trips so that they receive exactly the same preprocessing as the training data.

3.3 Reusability and Extensibility of the Pipeline

3.3.1 Reusable Components

The implementation is split into small, single-purpose modules, that don't contain tip-specific logic.

- **Training dataset generation.** The `training_set` job generates the initial training/validation/test split dataset based on JSON-configurable problem and table parameters, before storing it as a delta table.
- **Feature pipeline.** The preprocessing stages (median imputation, standardization, string indexing, one-hot encoding and vector assembly) are built by functions that use JSON-configurable feature and target columns. They can be applied to any tabular dataset.
- **Evaluation module.** `calculate_regression_metric` takes the label column, the prediction column and a metric name, so it works for any regression task. It also validates that the required columns exist before evaluating.
- **Training module.** This module is responsible for building the regressor, appending it to the feature pipeline, generating the final predictions, and saving the trained Spark ML pipeline model. It doesn't contain information regarding the context of the data it will handle.
- **Runner.** The main module loads the training Delta table, checks that all splits exist, fits the pipeline on the training split, evaluates it on the test split, and saves the model. Everything task-specific is read from the configuration file.
- **Configuration file.** A single JSON file defines the target column, the source table, the split column and the lists of numeric, categorical and datetime columns, as well as the model settings.

3.3.2 Incorporating New Features and Changing the Task

The pipeline is fully driven by the JSON configuration file, so adapting it to new features or a new prediction problem requires no code changes. The following changes are made in the configuration only:

- **Features.** Numeric columns can be added or removed through `numeric_columns`. Categorical columns must also be assigned to either `onehot_columns` or `ordinal_columns`. Datetime columns are added through `datetime_columns`, with their derived month and day features configured as ordinal columns. The pipeline then applies the corresponding transformations and adds the results to the final `features` vector.
- **Target.** The prediction target is changed by setting `target_column` (and `label_column`) to a different column.
- **Filtering logic.** The conditions that select which rows are used (for example, the payment type filter) are defined in the configuration and can be modified there.
- **Target validation logic.** The rules that decide whether a target value is valid (for example, requiring a non negative tip) are also set in the configuration.

Because every transformation is part of the same `Pipeline` object, newly added features are automatically fitted on the training split only, and the same transformations are applied to the validation and test data.

The only change that requires modifying code is replacing the model itself, or its evaluations metrics. The estimator is created in a single function (`build_regressor`), which currently returns a random forest. Using a different prediction algorithm means changing only that function. The feature pipeline, training, evaluation and model saving remain unchanged.

3.3.3 Retraining

When the enriched Gold Delta table is refreshed with new trips, the training-dataset generation job must first be run again. Running `train_tip_model_job.py` afterwards fits a new pipeline on the refreshed training dataset, recomputes the test metrics, and overwrites the saved model. Preprocessing statistics are re-estimated together with the model, so the two always stay consistent. The fixed seed makes runs reproducible for a given dataset.

3.3.4 Supporting Multiple Prediction Tasks

A new prediction task (for example, predicting the fare or the trip duration) can reuse the same code. It requires a new configuration file with a different target column, filters, feature lists, and output path, and, where needed, filtering conditions supported by the configuration. The training and runner modules stay unchanged because they read all names from the configuration.